Decision Tree Regression

Decision Tree Regression एक **Machine Learning algorithm** है जो **Regression problems (continuous value predict करने)** के लिए इस्तेमाल होता है।

इसमें डेटा को **पेड़ (Tree) की तरह छोटे-छोटे हिस्सों में बाँटा जाता है** और अंत में हर हिस्से में एक **prediction value** दी जाती है।

इसे Tree इसलिए कहते हैं क्योंकि इसकी structure ऐसी होती है:

```
                Age < 30
               /        \
           Yes           No
      Income < 50k     200
      /       \
  120         150
```

ऊपर:

- हर decision = **Node**
- शाखा = **Branch**
- अंतिम value = **Leaf**

Leaf node में **prediction value** होती है।

Decision Tree Regression

जब data **linear relation follow नहीं करता।**

Linear Regression:

```
y = mx + c
```

लेकिन कई बार data ऐसा होता है:

```
x < 10 → value = 50
x 10–20 → value = 100
x > 20 → value = 200
```

ऐसे **non-linear data** के लिए Decision Tree बहुत अच्छा काम करता है।

Decision Tree Mathematical

Algorithm **Mean Squared Error** कम करता है।

Formula:

$$MSE = \frac{1}{n}\sum(y_i - mean)^2$$

मतलब:

```
actual value - group mean
```

Regression tree में मुख्य goal होता है:

Error कम करना

इसके लिए **MSE (Mean Squared Error)** इस्तेमाल होता है।

Formula:

$$MSE = \frac{1}{n}\sum(y_i - \bar{y})^2$$

जहाँ

- y_i = actual value
- $\bar{y}$ = group का mean
- n = samples

Algorithm वो split चुनता है जिससे **MSE सबसे कम हो जाए।**

Example on formula

Suppose group:

```
salary = [30, 35, 40]
```

Mean:

```
35
```

MSE:

```
(30-35)² + (35-35)² + (40-35)²
= 25 + 0 + 25
= 50 / 3
= 16.67
```

Algorithm कोशिश करता है कि **MSE minimum हो।**

Scikit-learn

```python
from sklearn.tree import DecisionTreeRegressor

model = DecisionTreeRegressor()

model.fit(X, y)

prediction = model.predict([[5]])
```

गर control करना हो:

</> Python

```python
DecisionTreeRegressor(
    max_depth = 3,
    min_samples_split = 2,
    min_samples_leaf = 1
)
```

Linear Reg. VS Decision Tree

Linear Regression

```
straight line
```

Decision Tree

```
steps
```

Example

```
salary
80 |         ----
70 |        |
60 |        |
50 |    -----
40 |    |
30 |---
   ----------------
     experience
```

Advantages

- ✔ Non-linear data handle करता है
- ✔ समझना आसान
- ✔ scaling जरूरी नहीं

Disadvantages

- ✘ Overfitting जल्दी करता है
- ✘ prediction smooth नहीं होता

एक लाइन में समझो

Decision Tree Regression = data को question पूछकर छोटे groups में बांटता है और हर group का average prediction देता है।

Example

सोचिए आपको **किसी आदमी की** salary predict **करनी है**।

लेकिन formula लगाना मुश्किल है।

तो आप **questions पूछते जाते हैं**।

Example:

```
Experience < 5 ?
```

अगर **हाँ** → salary कम
अगर **नहीं** → salary ज्यादा

फिर अगला question।

यही **Decision Tree** है।

मतलब:

👉 Model **questions पूछकर** prediction **करता है**।

DataSet

Experience	Salary
1	30
2	35
3	40
6	70
7	80

अब model सोचता है:

सबसे अच्छा question क्या होगा?

मान लो question:

```
Experience < 4
```

Split the data

अब data दो भाग में बंट गया।

Left Side

```
1 → 30
2 → 35
3 → 40
```

Right Side

```
6 → 70
7 → 80
```

Leaf Node Prediction

अब algorithm हर group का **average लेता है**।

Left:

```
(30 + 35 + 40) / 3 = 35
```

Right:

```
(70 + 80) / 2 = 75
```

अब tree बन गया।

```
Experience < 4 ?

Yes → 35
No  → 75
```

Prediction

अगर input:

```
experience = 2
```

Tree पूछेगा:

```
2 < 4 ?
```

Yes

Prediction:

```
35
```

अगर input:

```
experience = 6
```

```
6 < 4 ?
```

No

Prediction:

```
75
```

Visualiztion

Visualization

```
        Experience < 4
        /            \
    Yes               No
  Salary=35       Salary=75
```

बस यही Decision Tree Regression है।

Algorithm Wants

Algorithm चाहता है कि:

```
हर group में values एक जैसी हों
```

Example:

Bad group

```
30, 80, 20, 90
```

Good group

```
30, 32, 35, 34
```

इसलिए algorithm **best split ढूंढता है।**

Parameters in Decision Tree

5 DecisionTreeRegressor में और कौन-कौनसे parameters पास किए जा सकते हैं?

1. `max_depth` → tree की maximum depth (लंबाई)
   - बहुत बड़ा depth → overfitting
   - छोटा depth → underfitting
2. `min_samples_split` → node split करने के लिए minimum samples
3. `min_samples_leaf` → leaf में minimum samples
4. `max_features` → split बनाने में कितने features consider किए जाएँ
5. `random_state` → reproducibility के लिए
6. `criterion` → splitting criterion (`'squared_error'`, `'friedman_mse'`, `'absolute_error'`, `'poisson'`)

All options of criterion

1️⃣ `squared_error` (पुराना `'mse'`)

- **पूरा नाम:** Mean Squared Error
- **कैसे काम करता है:**
  - हर split के लिए, यह **(actual value - predicted value)**^{2} की औसत निकालता है।
  - Tree वह split चुनता है जिससे **MSE सबसे कम** हो।
- **Use case:** सबसे सामान्य और default option है। Continuous target prediction में सबसे अच्छा।

2️⃣ `friedman_mse`

- **कैसे काम करता है:**
  - यह MSE का एक **optimized version** है जिसे Jerome Friedman ने design किया है।
  - Gradient boosting के लिए recommended।
- **Use case:** जब आप ensemble methods (जैसे Gradient Boosting) में tree का उपयोग कर रहे हों।

All options of criterion

3 `absolute_error`

- **कैसे काम करता है:**
  - यह split के लिए **Mean Absolute Error (MAE)** का उपयोग करता है।
  - MAE = औसत |actual - predicted|
- **Difference from MSE:**
  - MSE बड़े errors को ज्यादा punish करता है क्योंकि square होता है।
  - MAE बड़े errors को कम weight देता है → अधिक robust आउटliers के लिए।
- **Use case:** जब आपके data में outliers ज्यादा हों।

All options of criterion

4 `poisson`

- **कैसे काम करता है:**
  - यह **Poisson deviance** को minimize करता है।
  - Target variable को **count data** (जैसे events, incidents) के लिए design किया गया है।
- **Use case:** जब target केवल **positive integers** हो, जैसे मरीजों की संख्या, कॉल्स की संख्या आदि।

Summary

Criterion	Use Case	
squared_error	General regression	Most common, sensitive to outliers
friedman_mse	Gradient boosting	Optimized MSE
absolute_error	Regression with outliers	Less sensitive to large errors
poisson	Count data	Target ≥ 0, integer

Code

```
regressor = DecisionTreeRegressor(
    criterion='squared_error',
    max_depth=5,
    min_samples_split=10,
    random_state=42
)
regressor.fit(X_train, y_train)
score = regressor.score(X_test, y_test)
print(score)
```

What

- **Machine Learning** = Data + Learning + Prediction